Homework 4, Convex Optimization

Departamento de Matemáticas, Universidad de los Andes
Colombia

Student's name:	Luis Alejandro Rubiano
Student's code:	202013482
Email address:	la.rubiano@uniandes.edu.co
Class:	Convex Optimization
Instructor:	Mauricio Junca

1 Given $x \in C$ with C a convex set, let $N_C(x) := \{g : \langle g, y-x\rangle \leq 0, \forall y \in C\}$.

(a) Show that $N_C(x)$ is a closed convex cone.

Proof: Let $g_1, g_2 \in N_c(x)$ and $t > 0$. Then,

$$\begin{aligned} \langle tg_1, y-x\rangle &= t\langle g_1, y-x\rangle \text{ for all } y \in C \\ t\langle g_1, y-x\rangle &\leq 0 \text{ since } g_1 \in N_C(x) \text{ and } t > 0 \end{aligned}$$

Therefore, $tg_1 \in N_C(x)$. Also,

$$\begin{aligned} \langle g_1 + g_2, y-x\rangle &= \langle g_1, y-x\rangle + \langle g_2, y-x\rangle \text{ for all } y \in C \\ \langle g_1, y-x\rangle &\leq 0 \text{ since } g_1 \in N_C(x) \\ \langle g_2, y-x\rangle &\leq 0 \text{ since } g_2 \in N_C(x) \\ \langle g_1, y-x\rangle + \langle g_2, y-x\rangle &\leq 0 \end{aligned}$$

Therefore, $g_1 + g_2 \in N_C(x)$. Thus, $N_C(x)$ is a convex cone.

Finally, let g_n be a sequence in $N_C(x)$ that converges to g. Since for each $g_n \in N_C(x)$, we have that $\langle g_n, y-x\rangle \leq 0$ for all $y \in C$, then

$$\lim_{n\to\infty} \langle g_n, y-x\rangle \leq 0$$

for all $y \in C$. Because of the continuity of the inner product, we have that $N_C(x)$ is closed.

Hence, $N_C(x)$ is a closed convex cone. □

(b) Show that if $x \in \text{int}(C)$, then $N_C(x) = \{0\}$.

Proof: Let $x \in \text{int}(C)$, then there is some $\varepsilon > 0$ such that $B(x, \varepsilon) \subseteq C$. Consider the point $y := x + \varepsilon g$. Since $x \in \text{int}(C)$, then $y \in C$. Then,

$$\begin{aligned} \langle g, y-x\rangle &= \langle g, (x+\varepsilon g) - x\rangle \\ &= \langle g, \varepsilon g\rangle \\ &= \varepsilon \|g\|^2 \end{aligned}$$

Since $\varepsilon > 0$, then g must be the zero vector. Therefore, $N_C(x) = \{0\}$. $\square$

(c) Find the set for all the points of a closed line segment in $\mathbb{R}^2$.

Solution: Line segments in $\mathbb{R}^2$ have one of the following forms:

1. Case 1: $C_1 = \{(x,y) : x \in [x_0, x_1], y = mx + b\}$, $m, b, x_0, x_1 \in \mathbb{R}$, $x_0 < x_1$,
2. Case 2: $C_2 = \{(x,y) : x \in (x_0, x_1), y = mx + b\}$, $m, b, x_0, x_1 \in \mathbb{R}$, $x_0 < x_1$,
3. Case 3: $C_3 = \{(x,y) : x = c, y \in [y_0, y_1]\}$, $c, y_0, y_1 \in \mathbb{R}$, $y_0 < y_1$,
4. Case 4: $C_4 = \{(x,y) : x = c, y \in (y_0, y_1)\}$, $c, y_0, y_1 \in \mathbb{R}$. $y_0 < y_1$.

We will go through each case:

1. Case 1:

Let $z = (z_x, z_y) \in C_1$. If $z_x \in (x_0, x_1)$, then $N_{C_1}(z) = \{0\}$ because of (b). If $z = (x_0, mx_0 + b)$, then

$$\begin{aligned} N_{C_1}(z) &= \{g : \langle g, r - z\rangle \le 0, \forall r \in C_1\} \\ &= \{g : g_x(r_0 - x_0) + g_y((mr_0 + b) - (mx_0 + b)) \le 0, \forall r_0 \in [x_0, x_1]\} \\ &= \{g : g_x(r_0 - x_0) + g_y(m(r_0 - x_0)) \le 0, \forall r_0 \in [x_0, x_1]\} \\ &= \{g : g_x + mg_y \le 0\} \text{ since } r_0 - x_0 \ge 0 \end{aligned}$$

If $z = (x_1, mx_1 + b)$, then

$$\begin{aligned} N_{C_1}(z) &= \{g : \langle g, r - z\rangle \le 0, \forall r \in C_1\} \\ &= \{g : g_x(r_1 - x_1) + g_y((mr_1 + b) - (mx_1 + b)) \le 0, \forall r_1 \in [x_0, x_1]\} \\ &= \{g : g_x(r_1 - x_1) + g_y(m(r_1 - x_1)) \le 0, \forall r_1 \in [x_0, x_1]\} \\ &= \{g : g_x + mg_y \ge 0\} \text{ since } r_0 - x_0 \le 0 \end{aligned}$$

2. Case 2:

In this case $N_{C_2}(z) = \{0\}$ for all $z \in C_2$ because of (b).

3. Case 3:

Let $z = (z_x, z_y) \in C_3$. If $z_y \in (y_0, y_1)$, then $N_{C_3}(z) = \{0\}$ because of (b). If $z = (c, y_0)$, then

$$\begin{aligned} N_{C_3}(z) &= \{g : \langle g, r - z\rangle \le 0, \forall r \in C_3\} \\ &= \{g : g_x(c - c) + g_y(r_y - y_0) \le 0, \forall r_y \in [y_0, y_1]\} \\ &= \{g : g_y \le 0\} \text{ since } r_y - y_0 \ge 0 \end{aligned}$$

Likewise, if $z = (c, y_1)$, then $N_{C_3}(z) = \{g : g_y \ge 0\}$.

4. Case 4:

In this case $N_{C_4}(z) = \{0\}$ for all $z \in C_4$ because of (b). $\square$

2 ***Convex hull or envelope of a function.*** **The convex hull or envelope of a function $f : \mathbb{R}^n \to \mathbb{R}$ is defined as**

$$g(x) = \inf\{t \mid (x,t) \in \textbf{conv epi } f\}$$

Geometrically, the epigraph of g is the convex hull of the epigraph of f. Show that g is the largest convex underestimator of f. In other words, show that if h is convex and satisfies $h(x) \leq f(x)$ for all x, then $h(x) \leq g(x)$ for all x. Ex. 3.30. Convex Optimization, Boyd.

Solution: Let h be a convex function such that $h(x) \leq f(x)$ for all x. Since h is convex, **epi** h is a convex set. Since $h(x) \leq f(x)$ for all x, then **epi** $f \subseteq$ **epi** h. Therefore, **conv epi** $f =$ **epi** $g \subseteq$ **epi** h. Or equivalently, $g(x) \leq h(x)$ for all x. This is because the convex hull of **epi** f is the intersection of all convex sets that contain **epi** f. □

4 **Consider the function $f(x) = \langle c, x\rangle - \sum_{j=1}^{m} \log(1 - \langle a_j, x\rangle) - \sum_{i=1}^{n} \log(1 - x_i^2)$. Take $n = 5000$ and $m = 2000$ and vectors c and a_j chocen arbitrarily such that $\|a_j\| = 1$. For all methods start the iterations at $x_0 = 0$ and stopping criterion $\|\nabla f(x_k)\| \leq \varepsilon$. Graph $\log(f(x_k) - f^*)$ in each iteration (estimate f^* as best as possible) and compare the execution time of the methods.**

1. Use the quasi-Newton BFGS method with *backtracking* and Wolfe to chose the step size. Graph the condition number of H_k in each iteration.

Solution: This method took 3.79 hours to complete. Python3 code to solve this problem is given by:

```
import matplotlib.pyplot as plt
import os
from time import perf_counter
from numpy import float16, dot, ones, zeros, identity, log, outer, sqrt
from numpy.linalg import norm, solve, cond
import pandas as pd
from progress.bar import Bar
from random import sample

n = 5000
m = 2000
epsilon = float16(1e-6)
# C is the vector of ones
ONE = float16(1)
TWO = float16(2)
rho = float16(0.75)

f_optimal = -1264.24311

max_iterations = 20 # Maximum number of iterations
max_sub_iterations = 5 # Maximum number of sub-iterations

dots = {}
def optimized_dot(x,y):
    stringxy = str(x) + str(y)
    if stringxy in dots:
        return dots[stringxy]
    result = dot(x,y)
    dots[stringxy] = result
    return result

```

```
fs = {}
def f(x, A):
    stringx = str(x)
    if stringx in fs:
        return fs[stringx]

    result = sum(x)
    for j in range(m):
        result -= log(ONE - optimized_dot(A[j],x))
    for i in range(n):
        result -= log(ONE - x[i]*x[i])

    fs[stringx] = result
    return result

gradients = {}
def gradient(x, A):
    stringx = str(x)
    if stringx in gradients:
        return gradients[stringx]

    result = ones(n)
    with Bar('Gradient', fill='#', suffix='%(percent).1f%% - %(eta)ds', max=n) as bar:
        for i in range(n):
            for j in range(m):
                result[i] += A[j][i] / (ONE - optimized_dot(A[j],x))
            result[i] += TWO * x[i] / (ONE - x[i]*x[i])
            bar.next()

    gradients[stringx] = result
    return result

def graph(x, y, title, x_label, y_label, filename):
    fig = plt.figure()
    plt.plot(x, y)
    plt.title(title)
    plt.xlabel(x_label)
    plt.gca().xaxis.set_major_locator(plt.MaxNLocator(integer=True))
    plt.ylabel(y_label)
    filename = "images" + os.sep + filename + ".png"
    fig.savefig(filename, dpi=fig.dpi)
    plt.clf()

A = []
df = pd.read_csv('A.txt', header=None)
data = df.values
for j in range(m):
    # Save data[j] as numpy array of float16
    A.append(data[j].astype(float16))
print("A loaded")

## quasi-Newton BFGS with backtracking and wolfe conditions

time0 = perf_counter()


alpha = float16(0.15) # Initial alpha
c_1 = float16(10e-4)
c_2 = float16(0.9)
X = [zeros(n, dtype=float16)] # Initial point x_0 = 0
Fx = [f(X[-1],A)]
GRADIENTS = [gradient(X[-1], A)]
ALPHAS = [alpha]
H = [identity(n)]
P = [-H[-1] @ GRADIENTS[-1]]
H_condition_numbers = [cond(H[-1])]

iterations = 0


while (norm(GRADIENTS[-1]) > epsilon) and (iterations < max_iterations):
    print("Iteration", iterations)
    print("Condition number of H_k:", H_condition_numbers[-1])
    print("f(x_k):", Fx[-1])
    print("x_k:", X[-1])
```

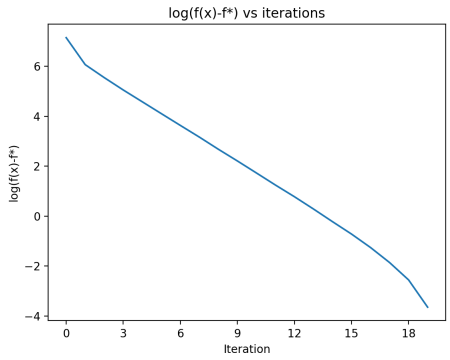

Figure 1: $\log(f(x)-f^*)$, using quasi-Newton BFGS method, $x_0 = 0$

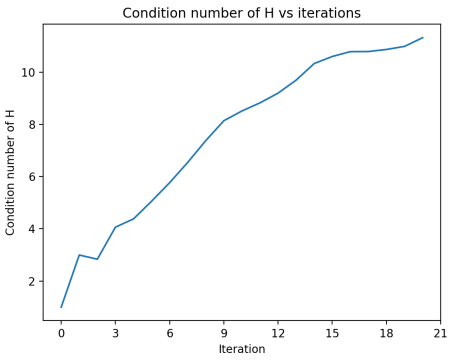

Figure 2: Condition number of H_k, using quasi-Newton BFGS method, $x_0 = 0$

```
    iterations += 1
    sub_iterations = 0

    while (f(X[-1] + ALPHAS[-1] * P[-1], A) > f(X[-1], A) + c_1 * ALPHAS[-1] * optimized_dot(
    gradient(X[-1], A), P[-1])
           or (- optimized_dot(gradient(X[-1] + ALPHAS[-1] * P[-1], A), P[-1]) > - c_2 *
    optimized_dot(gradient(X[-1], A), P[-1]))
           ) and sub_iterations < max_sub_iterations:
        sub_iterations += 1
        ALPHAS[-1] = rho * ALPHAS[-1]

    X.append(X[-1] + ALPHAS[-1] * P[-1]) # X_k+1 = X_k + alpha * P_k
    Fx.append(f(X[-1], A))
    GRADIENTS.append(gradient(X[-1], A))
    s = X[-1] - X[-2]
    y = GRADIENTS[-1] - GRADIENTS[-2]
    ALPHAS.append(alpha)
    H.append(H[-1] - (H[-1]@ (outer(y,y) @H[-1]))/(y.T @(H[-1]@y)) + outer(s,s)/(optimized_dot
    (y,s)))
    P.append(- H[-1] @ GRADIENTS[-1])
    H_condition_numbers.append(cond(H[-1]))

time1 = perf_counter()

print("Quasi-Newton BFGS with backtracking and wolfe conditions completed")
print("Iterations:", iterations)
print("Total time elapsed:", (time1 - time0)/3600, "hours")

log_Fx = [log(f - f_optimal) for f in Fx]
graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)",
    "4ai")
graph(range(iterations+1), H_condition_numbers, "Condition number of H vs iterations", "
    Iteration", "Condition number of H", "4aii")
```

2. Use the confidence region Newton method and the *dogleg* method to approximate solution to the sub-problem. Graph Δ_k in each iteration.

Solution: This method took 3.61 hours to complete. Python3 code to solve this problem is given by:

```
## Confidence region Newton method

dots = {}
fs = {}
gradients = {}
zero_vector = zeros(n)

def dogleg(grad, hess, rad):
    p_B = -solve(hess, grad)
    p_U = - optimized_dot(grad, grad) / (grad.T @ hess @ grad) * grad
```

```
    if norm(p_B) <= rad:
        return p_B
    elif norm(p_U) >= rad:
        return rad/norm(p_U) * p_U
    else:
        a = optimized_dot(p_B, p_B) - 2 * optimized_dot(p_B, p_U) + optimized_dot(p_U, p_U)
        b = 2 * optimized_dot(p_B, p_U) - 2 * optimized_dot(p_U, p_U)
        c = optimized_dot(p_U, p_U) - rad**2
        tau_minus_one = (-b + (b**2 - 4*a*c)**0.5) / (2*a)
        return p_U + tau_minus_one * (p_B - p_U)

def m_function_generator(fx, hess, grad):
    return lambda p: fx + optimized_dot(grad, p) + 0.5 * p.T @ hess @ p


def compute_rho(f, x, p, A, m):
    return (f(x, A) - f(x + p, A)) / (m(zero_vector)-m(p))


time2 = perf_counter()
X = [zeros(n, dtype=float16)] # Initial point x_0 = 0
Fx = [f(X[-1],A)]
DELTAS = [0.05] # Radius of the confidence region
Delta_max = 0.25
H = [identity(n)] # Initial Hessian
GRADIENTS = [gradient(X[-1], A)] # Initial gradient
eta = 0.25 # threshold
P = [dogleg(GRADIENTS[-1], H[-1], DELTAS[-1])] # Initial step
M = [m_function_generator(f(X[-1], A), H[-1], GRADIENTS[-1])] # Initial m function
Rho = [compute_rho(f, X[-1], P[-1], A, M[-1])] # Initial rho

iterations = 0

while (norm(GRADIENTS[-1]) > epsilon) and (iterations < max_iterations):
    print("Iteration", iterations)
    print("f(x_k):", f(X[-1],A))
    print("x_k:", X[-1])
    print("Delta_k:", DELTAS[-1])

    iterations += 1
    sub_iterations = 0

    if Rho[-1] < 0.25:
        DELTAS.append(DELTAS[-1] / 4)
    else:
        if Rho[-1] > 0.75 and norm(P[-1]) == DELTAS[-1]:
            DELTAS.append(min(2*DELTAS[-1], Delta_max))
        else:
            DELTAS.append(DELTAS[-1])

    if Rho[-1] > eta:
        X.append(X[-1] + P[-1])
    else:
        X.append(X[-1])

    # Delta and X were updated, now we update the other

    Fx.append(f(X[-1], A))
    GRADIENTS.append(gradient(X[-1], A))

    s = X[-1] - X[-2] #Update H by BFGS formula
    y = GRADIENTS[-1] - GRADIENTS[-2]
    H.append(H[-1] - (H[-1]@ (outer(y,y) @H[-1]))/(y.T @(H[-1]@y)) + outer(s,s)/(
  optimized_dot(y,s)))

    P.append(dogleg(GRADIENTS[-1], H[-1], DELTAS[-1]))
    M.append(m_function_generator(f(X[-1], A), H[-1], GRADIENTS[-1]))
    Rho.append(compute_rho(f, X[-1], P[-1], A, M[-1]))

time3 = perf_counter()

print("Confidence region Newton method completed")
print("Iterations:", iterations)
print("Total time elapsed:", (time3 - time2)/3600, "hours")
```

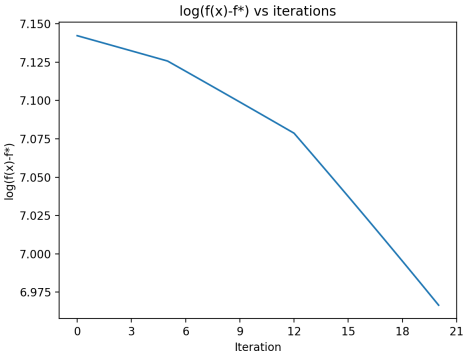

Figure 3: $\log(f(x) - f^*)$, using confidence region Newton method, $x_0 = 0$

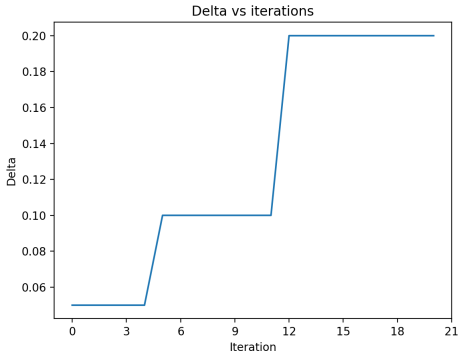

Figure 4: Δ_k, using confidence region Newton method, $x_0 = 0$

```
85
86    log_Fx = [log(f - f_optimal) for f in Fx]
87    graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)"
        , "4bi")
88    graph(range(iterations+1), DELTAS, "Delta vs iterations", "Iteration", "Delta", "4bii")
```

3. Use the accelerated Nesterov method with backtracking.

Solution: This method took 3.44 hours to complete. Python3 code to solve this problem is given by:

```
1  ## Accelerated Nesterov method with backtracking
2
3  dots = {}
4  fs = {}
5  gradients = {}
6  time4 = perf_counter()
7
8  X = [zeros(n, dtype=float16)] # Initial point x_0 = 0
9  Fx = [f(X[-1],A)]
10 lambda_prev = 0
11 lambda_curr = 1
12 gamma = 1
13 y_prev = X[-1]
14 alpha = 0.15 # Initial alpha
15 GRADIENTS = [gradient(X[-1], A)] # Initial gradient
16
17 iterations = 0
18
19 while (norm(GRADIENTS[-1]) > epsilon) and (iterations < max_iterations):
20     print("Iteration", iterations)
21     print("f(x_k):", Fx[-1])
22     print("x_k:", X[-1])
23
24     iterations += 1
25     sub_iterations = 0
26
27     y_curr = X[-1] - alpha * GRADIENTS[-1]
28     X.append((ONE - gamma) * y_curr + gamma * y_prev)
29     y_prev = y_curr
30
31     lambda_tmp = lambda_curr
32     lambda_curr = (1 + sqrt(1 + 4 * lambda_prev * lambda_prev)) / 2
33     lambda_prev = lambda_tmp
34
35     gamma = (1 - lambda_prev) / lambda_curr
36
37     Fx.append(f(X[-1], A))
38     GRADIENTS.append(gradient(X[-1], A))
39
40 time5 = perf_counter()
41
```

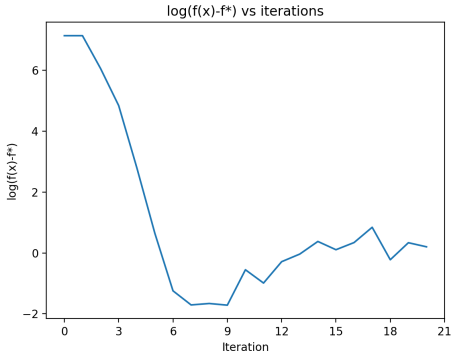

Figure 5: $\log(f(x) - f^*)$, using accelerated Nesterov method, $x_0 = 0$

```
print("Accelerated Nesterov method with backtracking completed")
print("Iterations:", iterations)
print("Total time elapsed:", (time5 - time4)/3600, "hours")

log_Fx = [log(f - f_optimal) for f in Fx]
graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)",
    "4ci")
```

4. Use the stochastic gradient method with *mini-batch*, that is, in each iteration choose only some terms in the sums to calculate the gradient, the number of terms chosen is the *mini-batch*. Change the *mini-batch* size and compare.

Solution: Here the mini-batch sizes are $500, 200$ and 100. They took 0.91, 0.36 and 0.18 hours to complete respectively. But in all cases, the method made the function get out of its domain. Python3 code to solve this problem is given by:

```
## Stochastic gradient descent with mini-batches of size m/4, m/10, m/20

def stochastic_gradient(x, A, batch):
    result = ones(n)
    with Bar('Gradient', fill='#', suffix='%(percent).1f%% - %(eta)ds', max=n) as bar:
        for i in range(n):
            for j in batch:
                result[i] += A[j][i] / (ONE - optimized_dot(A[j],x))
            result[i] += TWO * x[i] / (ONE - x[i]*x[i])
            bar.next()
    return result

def stochastic_gradient_method(batch_size):
    time_a = perf_counter()
    alpha = 0.15 # Initial alpha
    X = [zeros(n, dtype=float16)] # Initial point x_0 = 0
    Fx = [f(X[-1],A)]
    batch = sample(range(m), batch_size)
    batch.sort()
    STOCHASTIC_GRADIENTS = [stochastic_gradient(X[-1], A, batch)] # Initial gradient
    iterations = 0
    while (norm(STOCHASTIC_GRADIENTS[-1]) > epsilon) and (iterations < max_iterations):
        print("Iteration", iterations)
        print("f(x_k):", Fx[-1])
        print("x_k:", X[-1])
        print("Batch size:", batch_size)
        iterations += 1
        X.append(X[-1] - alpha * STOCHASTIC_GRADIENTS[-1])
        Fx.append(f(X[-1], A))
        batch = sample(range(m), batch_size)
        batch.sort()
        STOCHASTIC_GRADIENTS.append(stochastic_gradient(X[-1], A, batch))
    time_b = perf_counter()
    print("Stochastic gradient method with batch size", batch_size, "completed")
```

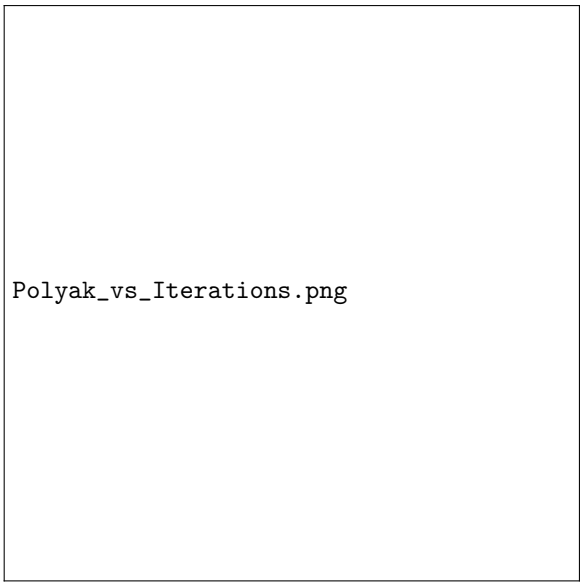

Figure 6: Polyak vs Iterations

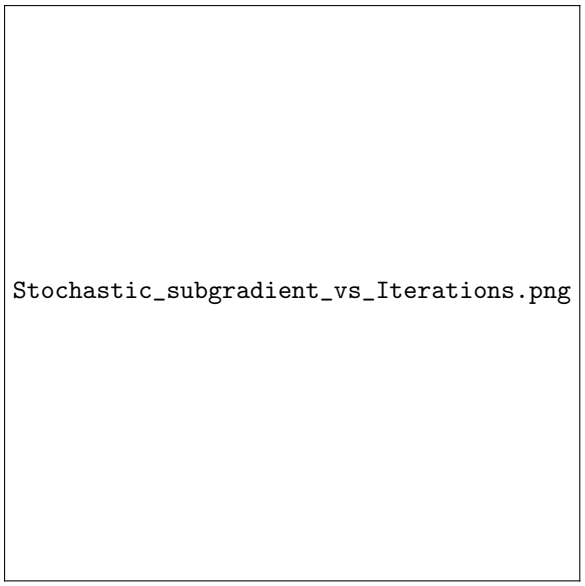

Figure 7: Stochastic subgradient vs Iterations

```
35      print("Iterations:", iterations)
36      print("Total time elapsed:", (time_b - time_a)/3600, "hours")
37      return Fx
38
39  dots = {}
40  fs = {}
41  batch_size = m//4 # Size of the mini-batches
42  Fx = stochastic_gradient_method(batch_size)
43  log_Fx = [log(f - f_optimal) for f in Fx]
44  graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)"
      , "4di")
45
46  dots = {}
47  fs = {}
48  batch_size = m//10 # Size of the mini-batches
49  Fx = stochastic_gradient_method(batch_size)
50  log_Fx = [log(f - f_optimal) for f in Fx]
51  graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)"
      , "4dii")
52
53  dots = {}
54  fs = {}
55  batch_size = m//20 # Size of the mini-batches
56  Fx = stochastic_gradient_method(batch_size)
57  log_Fx = [log(f - f_optimal) for f in Fx]
58  graph(range(iterations+1), log_Fx, "log(f(x)-f*) vs iterations", "Iteration", "log(f(x)-f*)"
      , "4diii")
```